0613-403 · Week 4 · Lecture 7

Filling a Shape: Boundary, Flood and Scan-Line Fill

Ekramul Haque Amin

Lecturer, AI and Data Science

Dhaka International University

Review: midpoint ellipse, a = 8, b = 4

Step 1: a^2 = ____, b^2 = ____

Step 2: $2b^2$ = ____, $2a^2$ = ____

Step 3: $p1_0 = b^2 - a^2b + \frac{1}{4}a^2$ = ____

Step 4: start (0, 4); $2a^2y$ = ____

Step 5: region 1 while $2b^2x < 2a^2y$

Step 6: $p2_0 = b^2(x + \frac{1}{2})^2 + a^2(y - 1)^2 - a^2b^2$ = ____

k	$p1_k$	Decision	(x_{k+1}, y_{k+1})	$2b^2x_{k+1}$	$2a^2y_{k+1}$

Region 2: $p2_0$ = ____ → choose ____ → pixel ____

Review solution: midpoint ellipse, a = 8, b = 4

Step 1: $a^2 = 64$, $b^2 = 16$

Step 2: $2b^2 = 32$, $2a^2 = 128$

Step 3: $p1_0 = 16 - 64(4) + 16 = -224$

Step 4: start $(0, 4)$; $2a^2y = 512$

Step 5: region 1 ends at $(8, 1)$: $256 \geq 128$

Step 6: $p2_0 = 16(8.5)^2 + 64(0)^2 - 1024 = 132$

k	$p1_k$	Decision	(x_{k+1}, y_{k+1})	$2b^2x_{k+1}$	$2a^2y_{k+1}$
0	-224	$p1 < 0 \rightarrow$ E	(1, 4)	32	512
1	-176	$p1 < 0 \rightarrow$ E	(2, 4)	64	512
2	-96	$p1 < 0 \rightarrow$ E	(3, 4)	96	512
3	16	$p1 \geq 0 \rightarrow$ SE	(4, 3)	128	384
4	-224	$p1 < 0 \rightarrow$ E	(5, 3)	160	384
5	-48	$p1 < 0 \rightarrow$ E	(6, 3)	192	384
6	160	$p1 \geq 0 \rightarrow$ SE	(7, 2)	224	256
7	144	$p1 \geq 0 \rightarrow$ SE	(8, 1)	256	128

Region 2: $p2_0 = 132 > 0 \rightarrow$ S $\rightarrow (8, 0)$. $y = 0$, stop.

Today's plan

Part 1

Seed fill

- 4- and 8-connected pixels
- Boundary fill
- Flood fill
- Leaks and stack overflow

Part 2

Scan-line polygon fill

- Fill row by row
- Edge intersections with 1/m
- The vertex rule
- Edge table and worked example

Part 2

Filling a shape

Boundary fill and flood fill

Two ways to fill a region

Seed fill (today)

Start at one pixel inside (the seed) and spread to its neighbours until the region is full.

- **Boundary fill**
- **Flood fill**

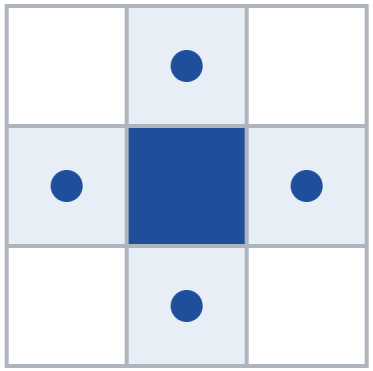
Used in paint programs: the bucket tool.

Scan-line fill (Part 2)

Go row by row; find where each row crosses the polygon edges and fill between the pairs.

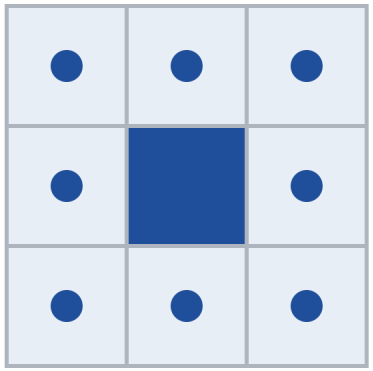
Used for polygons whose vertices we know.

Neighbours: 4-connected and 8-connected



4-connected

Right, left, up, down: (x±1, y) and (x, y±1).



8-connected

Also the 4 diagonals: (x±1, y±1).

Boundary fill

The region is closed by a boundary of one colour. Paint every pixel inside it.

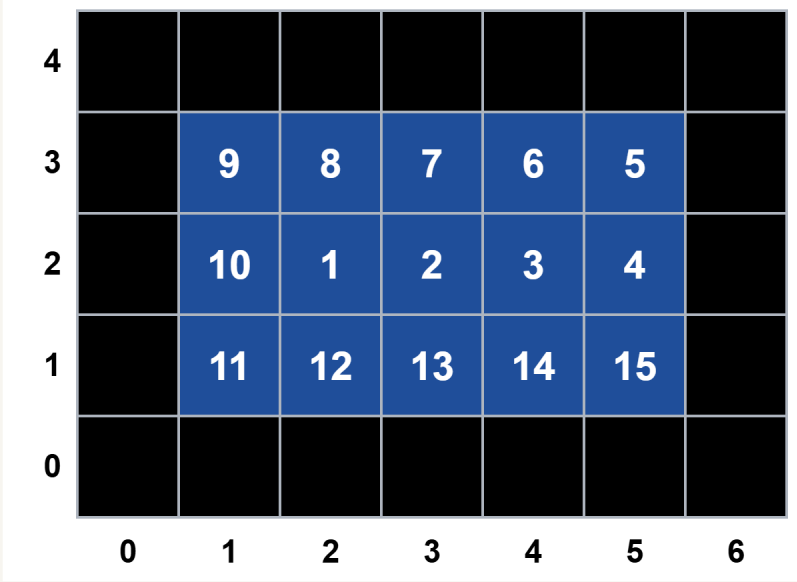
```
boundaryFill4(x, y, fill, boundary):
    c = getPixel(x, y)
    if c ≠ boundary and c ≠ fill:
        setPixel(x, y, fill)
        boundaryFill4(x + 1, y, fill, boundary)
        boundaryFill4(x - 1, y, fill, boundary)
        boundaryFill4(x, y + 1, fill, boundary)
        boundaryFill4(x, y - 1, fill, boundary)
```

Two stop rules

Stop at the boundary colour.

Stop at a pixel already painted with the fill colour, or we would loop for ever.

Boundary fill: the order pixels are painted



Black = boundary. Seed = (2, 2). Neighbour order: right, left, up, down.

The fill runs right as far as it can (1 → 4), then each call that returns tries its next direction.

All 15 inside pixels painted; the boundary is never crossed.

Flood fill

The region is all connected pixels of one old colour, whatever colour the border is. Replace that colour.

```
floodFill4(x, y, fill, old):
    if getPixel(x, y) == old:
        setPixel(x, y, fill)
        floodFill4(x + 1, y, fill, old)
        floodFill4(x - 1, y, fill, old)
        floodFill4(x, y + 1, fill, old)
        floodFill4(x, y - 1, fill, old)
```

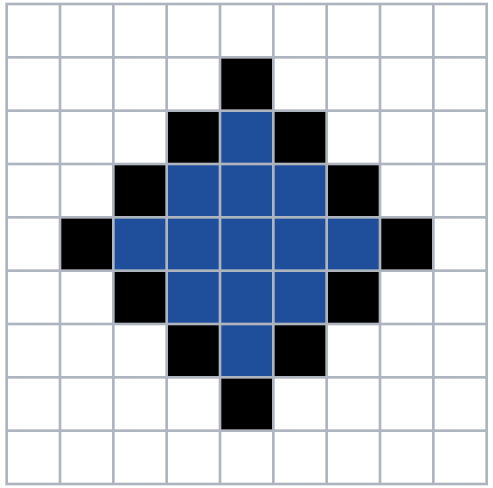
When to use it

The border has many colours (a map, a cartoon), so there is no single boundary colour to stop at.

Stop rule: the pixel is not the old colour.

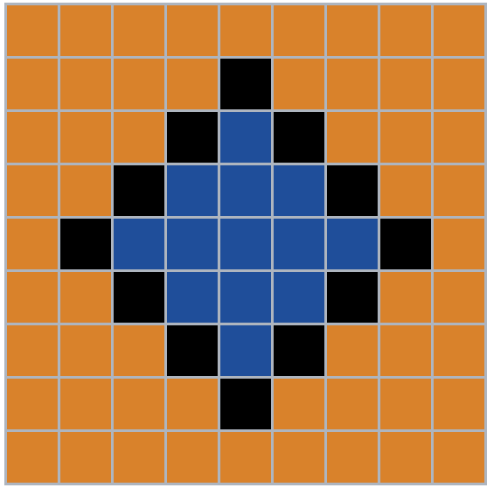
fill must differ from old, or it never stops.

Watch out: 8-connected fill can leak



4-connected

Stays inside: the diagonal steps of the border are enough to stop it.



8-connected

Slips between two border pixels that touch only at a corner and paints outside (orange).

Boundary fill and flood fill in C++

```
void boundaryFill4(int x, int y, Color fill, Color boundary) {
    Color c = getPixel(x, y);
    if (c ≠ boundary && c ≠ fill) {
        setPixel(x, y, fill);
        boundaryFill4(x + 1, y, fill, boundary);  boundaryFill4(x - 1, y, fill, boundary);
        boundaryFill4(x, y + 1, fill, boundary);  boundaryFill4(x, y - 1, fill, boundary);
    }
}
void floodFill4(int x, int y, Color fill, Color old) {
    if (getPixel(x, y) == old) {
        setPixel(x, y, fill);
        floodFill4(x + 1, y, fill, old);  floodFill4(x - 1, y, fill, old);
        floodFill4(x, y + 1, fill, old);  floodFill4(x, y - 1, fill, old);
    }
}
```

Problems with recursive seed fill



Deep recursion

One call per pixel. A 500 × 500 region can need 250,000 calls in a chain: the stack overflows (Python stops at about 1,000).



Repeated checks

Every pixel is tested up to 4 (or 8) times by its neighbours.



Fixes

- Use your own stack or queue instead of recursion
- Span fill: fill a whole row run at once, push only the rows above and below

Boundary fill vs flood fill

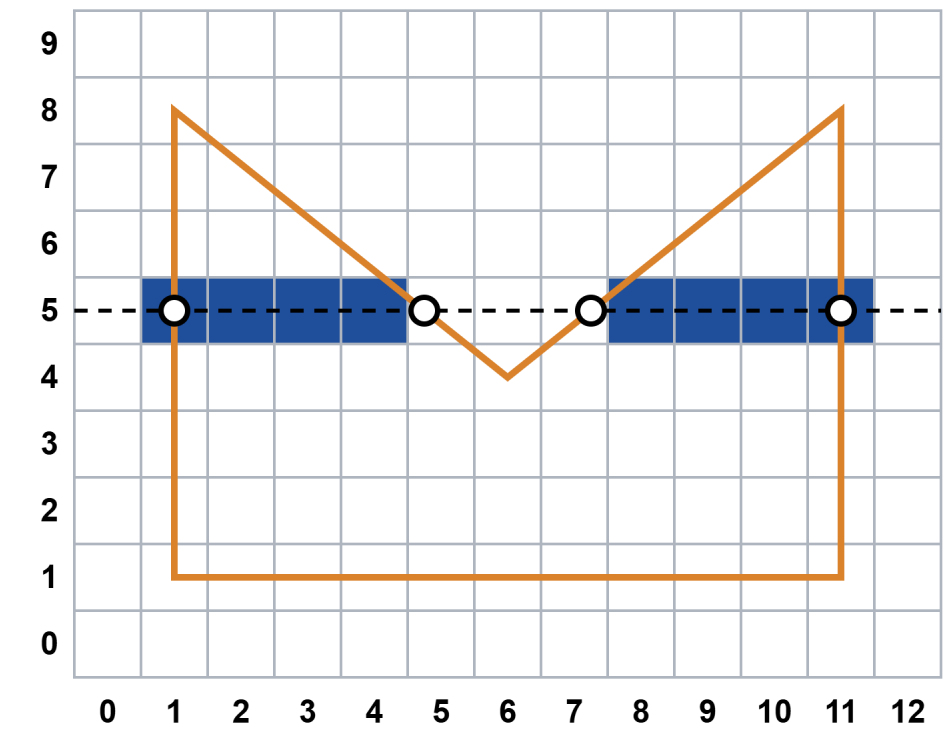
	Boundary fill	Flood fill
Region defined by	A border of one colour	Connected pixels of one old colour
Stops at	Boundary colour or fill colour	Any pixel not the old colour
Inputs	Seed, fill colour, boundary colour	Seed, fill colour, old colour
Border with many colours	Fails (leaks past other colours)	Works
Typical use	Filling a shape we just drew	Paint-bucket tool, maps, cartoons
Both	4- or 8-connected; recursive or with a stack	4- or 8-connected; recursive or with a stack

Part 2

Scan-line polygon fill

Fill a polygon one row at a time

The idea: fill one row at a time



For each scan line (row) y:

1. **Find where it crosses the polygon edges**
2. **Sort those x values**
3. **Fill between pairs: 1st–2nd, 3rd–4th, …**

Row y = 5: crossings 1, 4.75, 7.25, 11 → fill 1–4 and 8–11

Between a pair we are inside; between pairs we are outside.

Finding the crossings: x goes up by 1/m

Edge from (x1, y1) to (x2, y2), slope m = (y2 − y1) ÷ (x2 − x1).

Moving up one scan line, y goes up by 1, so x changes by 1/m:

$$x_{k+1} = x_k + 1/m,\ 1/m = (x2 - x1) \div (y2 - y1)$$

Start at the lower end (ymin) with x = x at ymin.

Example: edge (6, 4) to (11, 8): 1/m = 5 ÷ 4 = 1.25

y = 4: x = 6; y = 5: 7.25; y = 6: 8.5; y = 7: 9.75

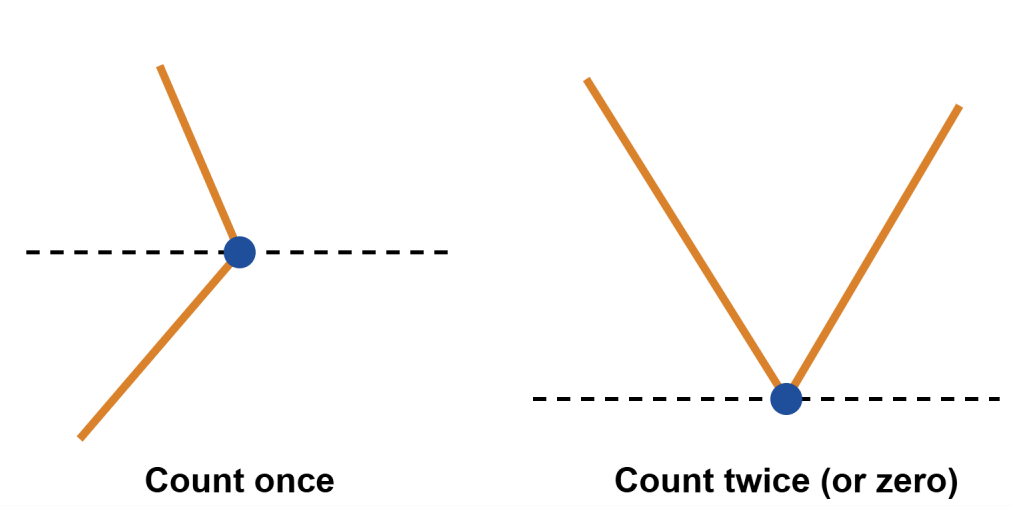
Same idea as DDA

No multiplication per row: just add 1/m.

Vertical edge: 1/m = 0, x never changes.

Horizontal edge: skip it, its top and bottom are the same row.

The vertex rule



A scan line through a vertex touches two edges at one point. How many crossings is that?

Edges on opposite sides

One goes up, one goes down: count 1.

Edges on the same side

A peak or valley: count 2 (or 0).

Easy way: each edge covers rows ymin ≤ y < ymax (include the bottom, exclude the top). This gives the right count automatically.

Edge table and active edge list

Edge table (ET)

Built once, before filling. Each non-horizontal edge is stored in the bucket of its ymin:

(ymax, x at ymin, 1/m)

Horizontal edges are left out.

Active edge list (AEL)

The edges that cross the current row:

1. Add edges from the ET bucket for this y
2. Remove edges with ymax = y
3. Sort by x, fill between pairs
4. Add 1/m to every x, go to y + 1

Scan-line fill algorithm

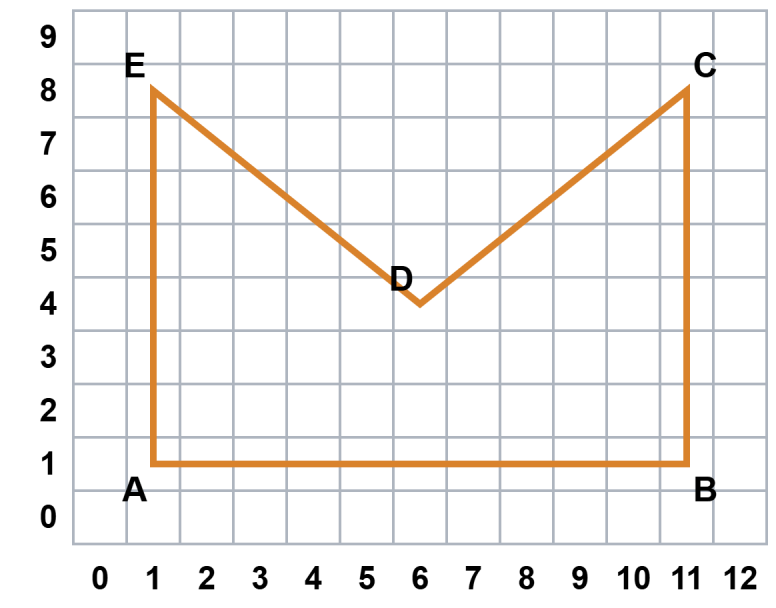
1. Build the edge table: for each non-horizontal edge store (ymax, x at ymin, 1/m) in bucket ymin.
2. y = the smallest ymin; the active edge list is empty.
3. Move the edges in bucket y into the active list.
4. Remove active edges with ymax = y.
5. Sort the active edges by x. Fill pixels from $\lceil x_1 \rceil$ to $\lfloor x_2 \rfloor$, from $\lceil x_3 \rceil$ to $\lfloor x_4 \rfloor$, ...
6. For every active edge: $x = x + 1/m$. Then $y = y + 1$.
7. Repeat steps 3–6 until the edge table and the active list are both empty.

Example: polygon A(1,1) B(11,1) C(11,8) D(6,4) E(1,8)

AB is horizontal: skip it. The other four edges go into the edge table:

Bucket ymin	Edge	ymax	x at ymin	1/m
1	EA (1,1)–(1,8)	8	1	0
1	BC (11,1)–(11,8)	8	11	0
4	DE (6,4)–(1,8)	8	6	-1.25
4	CD (6,4)–(11,8)	8	6	1.25

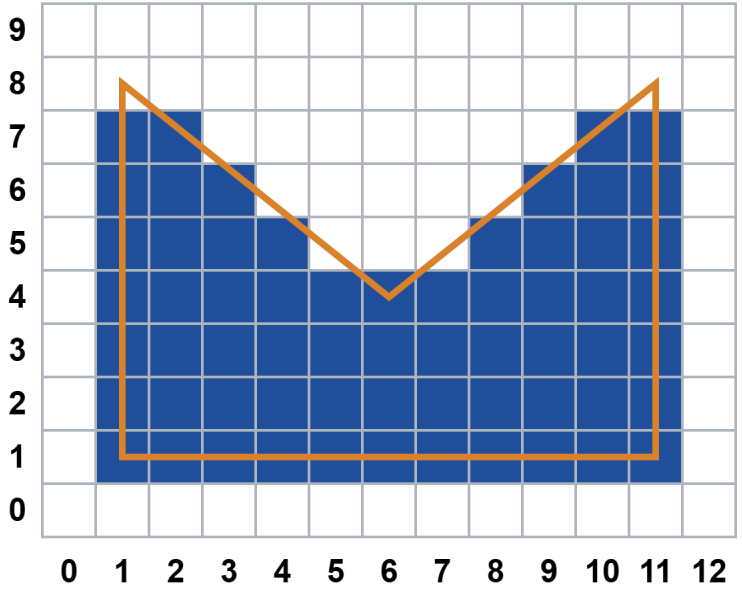
D is a valley: DE and CD both start at y = 4, so the row through D gets two crossings there.



Example: filling row by row

y	Active edges (sorted x)	Pairs	Pixels filled
1	1, 11	[1, 11]	x = 1–11
2	1, 11	[1, 11]	x = 1–11
3	1, 11	[1, 11]	x = 1–11
4	1, 6, 6, 11	[1, 6] [6, 11]	x = 1–6, x = 6–11
5	1, 4.75, 7.25, 11	[1, 4.75] [7.25, 11]	x = 1–4, x = 8–11
6	1, 3.5, 8.5, 11	[1, 3.5] [8.5, 11]	x = 1–3, x = 9–11
7	1, 2.25, 9.75, 11	[1, 2.25] [9.75, 11]	x = 1–2, x = 10–11

y = 8 is every edge's ymax, so nothing is active: stop.



Scan-line fill in C++ (simplified)

```
struct Edge { int ymax; float x, inv; }; // inv = 1/m
void scanFill(vector<Edge> ET[], int yMin, int yMax) {
vector<Edge> AEL;
for (int y = yMin; y < yMax; y++) {
for (Edge e : ET[y]) AEL.push_back(e); // add new edges
AEL.erase(remove_if(AEL.begin(), AEL.end(),
[y](Edge e) { return e.ymax == y; }), AEL.end()); // drop finished
sort(AEL.begin(), AEL.end(),
[](Edge a, Edge b) { return a.x < b.x; });
for (int i = 0; i + 1 < AEL.size(); i += 2) // pairs
for (int x = ceil(AEL[i].x); x ⩽ floor(AEL[i+1].x); x++)
setPixel(x, y);
for (Edge &e : AEL) e.x += e.inv; // next row
}
}
```

Seed fill vs scan-line fill

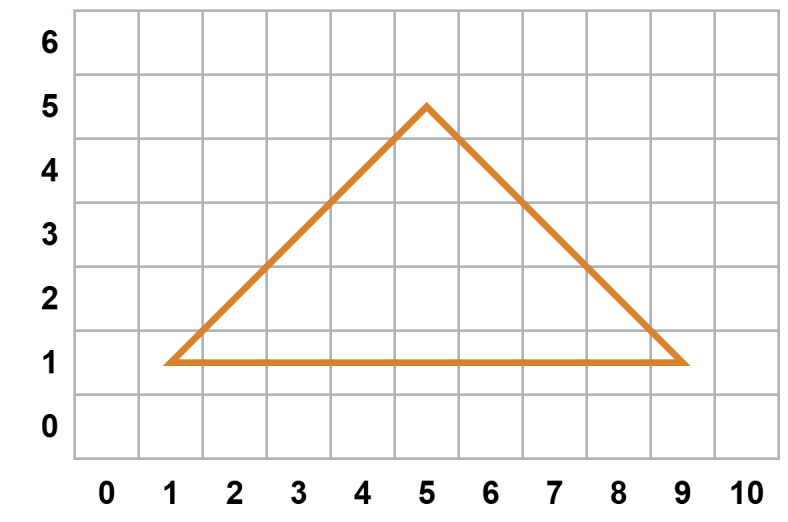
	Seed fill (boundary / flood)	Scan-line fill
Input	A seed pixel inside, colours on screen	The polygon's vertex list
Works	Pixel by pixel, spreading to neighbours	Row by row, between edge crossings
Memory	Big stack for big regions	Only the edge table and active list
Speed	Slower: each pixel tested up to 4–8 times	Faster: each pixel set once
Shapes	Any closed shape already drawn	Polygons (also concave)
Used in	Paint-bucket tools	Graphics hardware and polygon rendering

Class exercise: your turn

Scan-line fill the triangle (1, 1), (9, 1), (5, 5).

1. Which edge is skipped?
2. Edge table: ymax, x at ymin, 1/m
3. Row by row: crossings and pixels
4. Shade the pixels

Time: 6 minutes.

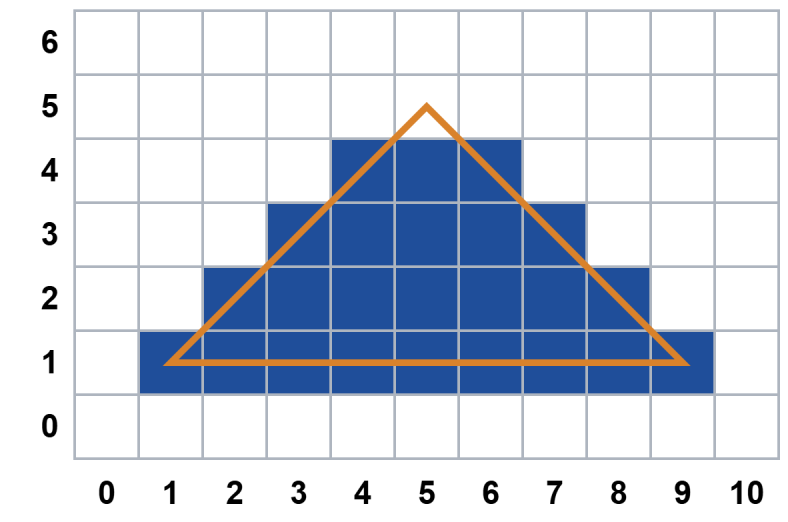


Exercise answer

Skip (1,1)–(9,1): horizontal.

Edge	ymax	x at ymin	1/m
(1,1)–(5,5)	5	1	1
(9,1)–(5,5)	5	9	−1

y	Crossings	Pixels filled
1	1, 9	x = 1–9
2	2, 8	x = 2–8
3	3, 7	x = 3–7
4	4, 6	x = 4–6



y = 5 is ymax for both edges: stop. The top vertex is not painted (half-open rule).

Key points from today

1. Seed fill spreads from one inside pixel to its 4 or 8 neighbours.
2. Boundary fill stops at one border colour; flood fill replaces one old colour.
3. 8-connected fill can leak through diagonal gaps; recursion can overflow the stack.
4. Scan-line fill: for each row, find the edge crossings, sort them, fill between pairs.
5. Crossings move by 1/m per row, like DDA; horizontal edges are skipped.
6. Each edge covers ymin ≤ y < ymax: this handles the vertex cases.
7. Edge table: (ymax, x at ymin, 1/m) by ymin; the active list holds the current edges.

Practice questions

1. Write the 4-connected boundary fill algorithm. How is flood fill different?
2. Draw a border where an 8-connected fill leaks but a 4-connected fill does not.
3. Why is recursive flood fill risky for a large region? Give one fix.
4. Scan-line fill the triangle (2, 2), (12, 2), (7, 7). Show the edge table and every row.
5. Explain the vertex rule with a drawing of a peak, a valley and a pass-through vertex.
6. Give three differences between seed fill and scan-line fill.

Coming up

Next lecture

2D transformations

Translation, rotation and scaling of points and shapes, with simple formulas and hand examples

Next lab

Fill it yourself

Flood fill with your own stack, then scan-line fill of a polygon, in Python with PyOpenGL

Questions?

Thank you

Ekramul Haque Amin · Lecturer, AI and Data Science